Problem 1 (25 pts)

- Snowfall

You are given an array A[1…n] which contains the cumulative snowfall totals for n consecutive days. For example, if there are 6 inches on snow on day 1, 7 inches of snow on day 2, 0 inches of snow on day 4 and 4 inches of snow on day 4 the array would contain the values [6,13,13,17].

You should read an input file from standard in which will contain 2 lines, the first line will be the number of days and the second line will be the a comma separated set of cumulative snowfall totals

Design and implement an algorithm that determines if there exists three consecutive days that produced more than half of the total snowfall across the n days.

Your solution should return either YES or NO.

Problem 1 con't (25 pts)

- Snowfall

You should demonstrate your code as run with the sample input on the following slide. You may run all of the inputs in a single execution of your code or individually. Your output should be structured as the input you are testing followed by your solution.

- Describe the heart of your algorithm
- Write a program that takes as input a file and returns to the console "**<input> solution: <yes|no>**"
- Provide a readme.txt which explains how to run your code in a terminal window.

Problem 1 con't (25 pts)

- Snowfall

Note: there are four sample inputs: The form of the input is the number of days on line1 followed by a space separated list of cumulative snowfall for those days:

8
1 4 10 14 15 17 17 23

16
44 127 166 231 238 320 363 429 500 511 577 642 689 764 812 886

9
3 8 14 19 21 24 25 25 32

5
1 2 3 4 5

Problem 1 con't (25 pts)

- Snowfall

Results:

1 4 10 14 15 17 17 23 solution: YES

44 127 166 231 238 320 363 429 500 511 577 642 689 764 812 886 solution: NO

3 8 14 19 21 24 25 25 32 solution: NO

1 2 3 4 5 solution: YES

Problem 2 (25 pts)

- Pandemic

You are in a square state with 25 counties. The counties can be arranged into an NxN grid (for our inputs 5x5). Two counties are neighbors if they are horizontally or vertically next to each other.

e.g.

	A	**B**		
		C		

A and B are neighbors. B and C are neighbors. A and C are not neighbors

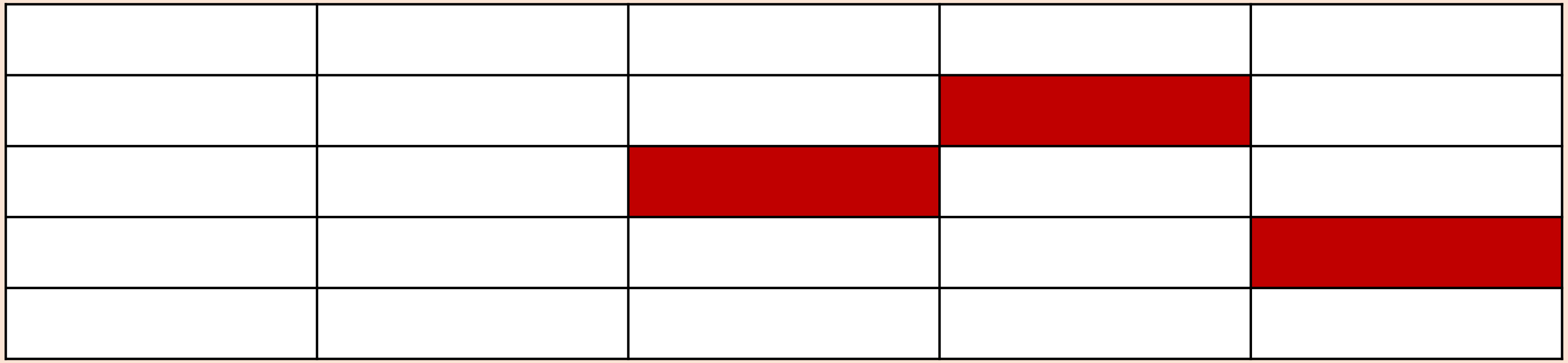
Problem 2 con’t (25 pts)

- Pandemic con’t

Some of the counties contain people infected by a mysterious virus which is highly contagious. If a county has an infected person, that person never becomes healthy and the county is considered infected. Each day the infection spreads to healthy counties if that county has at least two infected neighbors.

For example let’s start with the following configuration of infected (in red) counties:

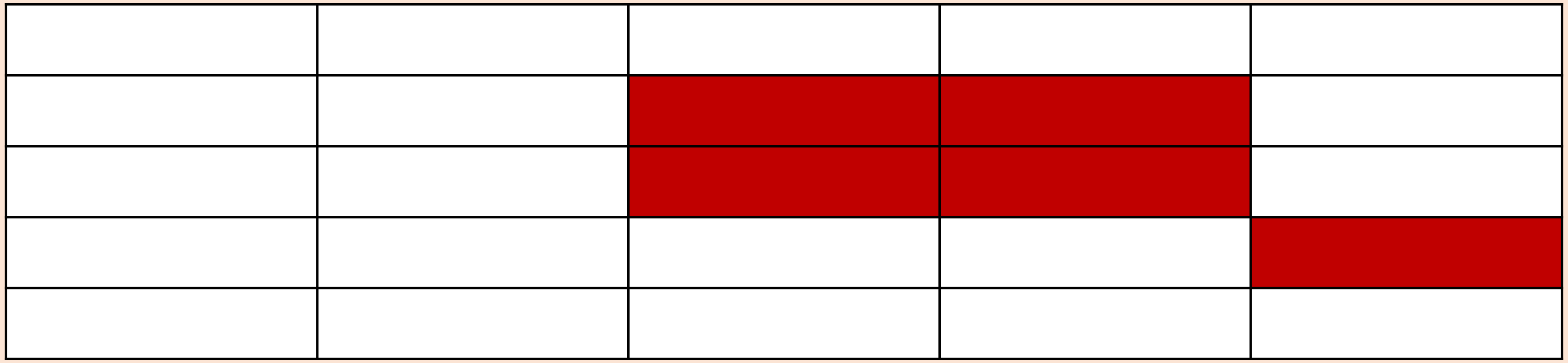
Day 1



Problem 2 con't (25 pts)

- Pandemic con't

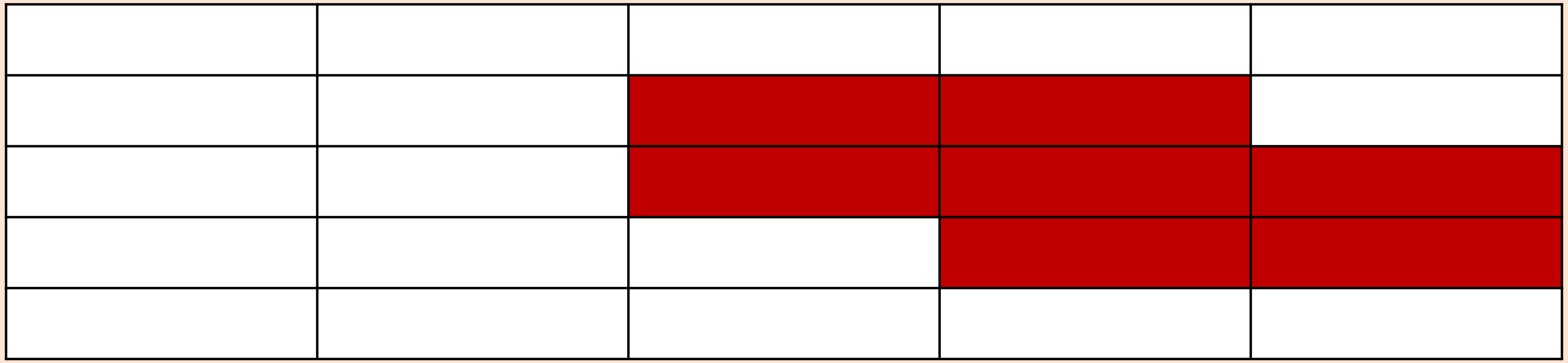
Day 2



Problem 2 con't (25 pts)

- Pandemic con't

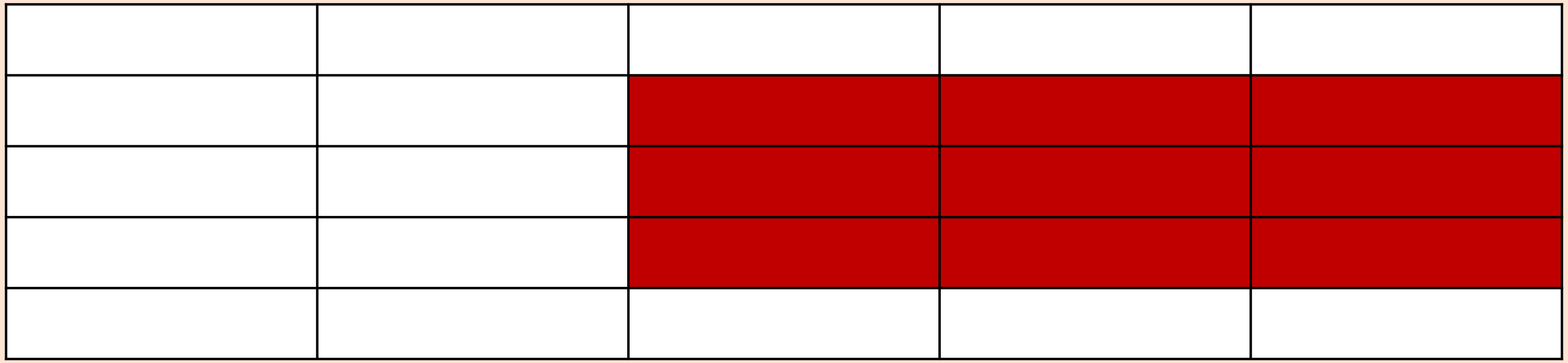
Day 3



Problem 2 con’t (25 pts)

- Pandemic con’t

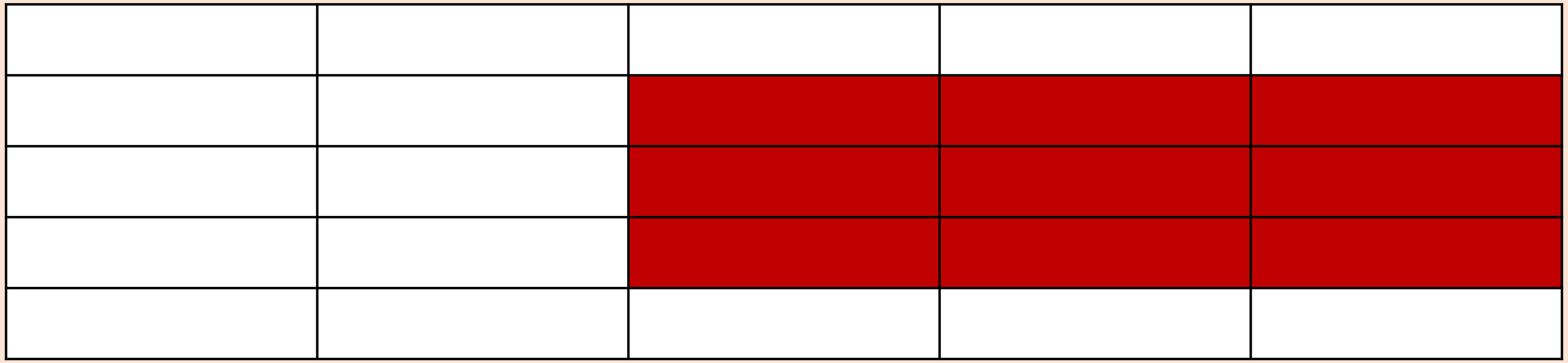
Day 4



Problem 2 con't (25 pts)

- Pandemic con't

Day 5 – The infection has stopped spreading



Problem 2 con't (25 pts)

- Pandemic con't

Write an algorithm to calculate if the infection will stop spreading and there are any healthy counties left in the state.

You will be given 2 input files which have the form:

Size of matrix

List of coordinate cells of initially infected counties

e.g.

5
1 3
2 2
3 4

This input means that the input matrix is 5x5 and cells (1,3), (2,2), (3,4) are initially infected

Problem 2 con’t (25 pts)

- Pandemic con’t
  - Describe the heart of your algorithm
  - Write a program that takes as input a file and returns to the console “**There <are|are no> healthy counties lef**t”
  - Provide a readme.txt which explains how to run your code in a terminal window.

Note: there are two sample inputs:

pandemic_input1.txt -> There are healthy counties left

pandemic_input2.txt -> There are no healthy counties left

Problem 3 (25 pts)

- Shopping Cart

You are going shopping at your local store. The aisles are organized such that each aisle contains a specific category of item. The aisles are arranged from left to right. The aisles are represented by an array of categories e.g.

['dinner', 'lunch', 'breakfast', 'snacks', 'desserts']

You want to collect as many items as you can moving through the aisles from left to right. The store owner has some very strict rules for the shoppers:

- You have only 2 baskets to place your items in and each basket can hold only a single category of item.
- Starting from the aisle of your choice, you must select exactly one item from the aisle and place it in one of your baskets, remember that you have only 2 baskets and each basket can contain only one category of item.
- Once you reach an aisle that contains an item you are not allowed to put into either of your baskets you must stop shopping.

Problem 3 (25 pts)

- Shopping Cart

You will be given input files with the first line containing the number of aisles and line two containing a comma separated list of categories

e.g. file 1:

3

dinner,lunch,dinner

e.g. file 2:

5

dinner,lunch,breakfast,lunch,lunch

Write an algorithm which will return the maximum number of items you can select from the given aisle category input as “<items> items were selected”.

e.g. Input: [‘dinner’, ‘lunch’, ‘dinner’] -> 3 items were selected

e.g. Input: [‘dinner’, ‘lunch’, ‘breakfast’, ‘lunch’, ‘lunch’] -> 4 items were selected

Problem 3 con’t (25 pts)

- Shopping Cart Con’t
  - Describe the heart of your algorithm and explain your code in a readme.txt file
  - Write a program that takes as input a file and returns to the console **“<# of items> items were selected”**
  - Provide a readme.txt which explains how to run your code in a terminal window.

Problem 4 (25 pts)

- Symbol Puzzle Game

You are given an nxn puzzle board where the square root of n is an integer e.g. 4x4 or 9x9 etc. The board is populated with symbols where the number of symbols is equal to n.

Example: A partially filled 4x4 puzzle board with symbols: [@#&*] with a . representing a non-filled position.

@	.	.	.
.	*	.	#
#	.	*	&
*	&	.	@

Problem 4 con’t (25 pts)

- Symbol Puzzle Game Con’t

Create and code an algorithm to calculate if the current board is in a valid or invalid state. The board is considered valid if the following conditions are met:

- Each row must contain each symbol at most once
- Each column must contain each symbol at most once
- The board contains sub-boards of size square root of n and each sub-board must contain each symbol at most once

Note: there are two sample inputs:

spg_input1.txt -> The board is valid

spg_input2.txt -> The board is invalid

Problem 4 con't (25 pts)

- Symbol Puzzle Game Con't

You will be provided 2 input files with the following structure:

The first row will be n, the second row will contain a comma separated list of the symbols and each subsequent row will be some combination of symbols and dots (representing empty cells) comma separated:

```
4
@,*,&,#
@,.,.,.
.,*,.,#
#,.,*,&
*,&,.,@
```

Note: there are two sample inputs:

spg_input1.txt -> The board is valid

spg_input2.txt -> The board is invalid

Problem 4 con’t (25 pts)

- Symbol Puzzle Game Con’t

  - Describe the heart of your algorithm, explain your code in a readme.txt
  - Write a program that takes as input a file and returns to the console “**The board is <valid|invalid>**”
  - Provide a readme.txt which explains how to run your code in a terminal window.

Note: there are two sample inputs:

spg_input1.txt -> The board is valid

spg_input2.txt -> The board is invalid